

RESEARCH ARTICLE

Automatically Seed Corpus and Fuzzing Executables Generation Using Test Framework

SANGHOON JEON^{1,2}, MINSOO RYU¹, DONGYOUNG KIM¹,
AND HUY KANG KIM¹, (Member, IEEE)

¹School of Cybersecurity, Korea University, Seoul 02841, Republic of Korea

²Samsung Electronics, Suwon-si 16677, Republic of Korea

Corresponding author: Huy Kang Kim (cenda@korea.ac.kr)

This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) funded by the Ministry of Science and ICT of the Korean Government (MSIT), and in part by the Development of Intelligence Cyber Attack and Defense Analysis Framework for Increasing Security Level of Cooperative Intelligent Transport Systems (C-ITS) under Grant 2021-0-00624.

ABSTRACT Fuzzing is widely utilized as a practical test method to determine unknown vulnerabilities in software. Although fuzzing shows excellent results for code coverage and crash count, it is not easy to apply these effects to library fuzzing. A library cannot run independently; it is only executed by an application called a customer program. In particular, a fuzzing executable and a seed corpus are needed to execute the library code by calling a specific function sequence and passing the input of the fuzzer to reproduce the various states of the library. However, preparing the environment for library fuzzing is challenging because it relies on the human expertise and requires both an understanding of the library and fuzzing knowledge. This study proposes *FuzzBuilderEx*, a system that provides an automated fuzzing environment for a library by utilizing the test framework to resolve this problem. *FuzzBuilderEx* conducts a static/dynamic analysis of the test code to automatically generate seed corpus and fuzzing executables that enable library fuzzing. Furthermore, the automatically generated seed corpus and fuzzing executable are compatible with existing fuzzers, such as the American Fuzzy Lop (AFL). This study applied *FuzzBuilderEx* to nine open-source libraries for performance evaluation and confirmed the effects of an increase in code coverage by 31.2% and a unique crash count of 58.7% compared to previous studies. Notably, we detected three zero-day vulnerabilities and registered one of them in the common vulnerabilities and exposures (CVE) database.

INDEX TERMS Data-driven security, fuzzing, seed generation, vulnerability detection, test framework.

I. INTRODUCTION

Fuzzing is widely used today as one of the most effective testing techniques for discovering security vulnerabilities in software. The Fuzzer executes the program with a randomly generated value as the input and monitors the behavior for invalid operations such as memory corruption issues. Programs can generate complex program states as each input value changes during fuzzing to reach a different program path. Since Miller *et al.* [1] first proposed black-box fuzzing, recent advances in grey-box fuzzing technologies such as AFL [2], AFLFast [3], and CollaAFL [4] have enabled

fuzzing to discover even more bugs by reaching deeper paths. Moreover, fuzzing efficiency is continually increasing by hybrid fuzzers such as T-Fuzz [5], QSYM [6], and Angora [7] based on execution feedback and code coverage.

Although this fuzzing technique shows excellent results for code coverage and unique crash counts found in the program, it is not easy to apply its effects to library fuzzing. Libraries, by their nature, only expose application programming interfaces (APIs) without dependency information between individual functions. Functions must be called in the correct order using the proper parameters to build a complex state shared between function calls. Because libraries explicitly do not describe the dependencies between functions, an efficient fuzzing campaign may not occur if a fuzzer randomly calls a

The associate editor coordinating the review of this manuscript and approving it for publication was Jiafeng Xie.

function using a random parameter. Thus, a typical approach for effectively fuzzing a library is to manually write a minimal program that establishes an initial state and calls API functions in a valid sequence. This approach enables a fuzzer to build various complex states necessary for testing library functions. However, to manually write a minimal program for fuzzing, a tester must understand the codebase under testing and the operating mechanism in a fuzzer's engine. In other words, the API call/dependency and fuzzing parameters are determined entirely by the tester's knowledge. Moreover, this labor-intensive work incurs maintenance costs according to changes in the library's code.

Libfuzzer [8] facilitates the fuzzing of libraries by testers. The tester calls various library functions to set a specific state and creates a fuzzer stub that utilizes arbitrary inputs to control the flow. Testers should implement these fuzzer stubs for each test unit. It is entirely at the tester's discretion to determine the API call, API dependencies, and parameters, which are components of the fuzz stub. Therefore, this approach cannot be extended to various libraries because it still relies on deep human knowledge. OSSFuzz [9] supports continuous fuzzing for open-source projects. Notably, it has contributed to improving the quality of open-source projects by supporting massive automated fuzzing. However, for applying the target project to OSSFuzz, the tester should manually prepare fuzzing executables and seed corpora.

To resolve these problems, research on the automation of library fuzzing is being conducted. FUDGE [10] and FuzzGen [11] have proposed a method to analyze consumer programs that use a library to generate a fuzz driver running the library. This method can be applied to set the initial state of the fuzz driver and call the library's API sequence. Because this method generates the target library by analyzing consumer programs, there are some disadvantages. Limited fuzzing may be performed for only some functions, rather than all states and API sequences, depending on the consumer program's characteristics. It is also difficult to configure an independent system because the library fuzz driver is generated by analyzing the consumer program. Moreover, it is challenging to minimize human expert interventions because the seed corpus still needs to be manually generated.

Our prior study proposed FuzzBuilder [12], a system that automatically generates a seed corpus and fuzzing executable for library fuzzing based on the test framework to address the above limitations. FuzzBuilder showed significant results on libraries that were difficult to apply to fuzz and gather seeds. However, this previous system had some limitations, such as the manual selection of the fuzzable API (FA) for the target library, low efficiency, and limited coverage of the generated seed corpus and fuzzing executables. Thus, the present study proposes *FuzzBuilderEx*, a system that dramatically overcomes the limitations of the previous system. The differences from the previous study are described as follows in the following three aspects: First, this study improved the level of automation for library fuzzing by eliminating the steps that partially require human interventions.

Second, this study increased fuzzing coverage by extending the fuzzing target function, which was limited by specific criteria (only string-type parameters). Third, we solved the problem of low fuzzing speed that automatically generated fuzzing executables based on the test code may have by removing unnecessary functions or code statements and creating the fuzzing executable in each detailed function unit.

FuzzBuilderEx is based on the following intuitions. First and foremost, test scenarios applied to the test framework contain critical paths that developers must validate during development. It is possible to analyze these critical paths for vulnerability detection through fuzzing. Second, the test framework automatically generates executable files that contain a critical path. The fuzzing executable can be automatically created by attaching a fuzz engine to these executables. Finally, the test data applied to the test framework are the input data that can reach the critical path. These test data can be automatically created as a seed corpus for fuzzing during test binary running. For the above reasons, this study uses the test framework to solve various limitations that make it challenging to apply fuzzing to the library.

The *FuzzBuilderEx* proposed in this study automatically generates the seed corpus and fuzzing executable by utilizing the information obtained through static/dynamic analysis of the test code existing in the library project. *FuzzBuilderEx* frees the tester from the burden of writing code for library fuzzing and allows the tester to easily apply fuzzing to the library without understanding the library. The contributions of this study are as follows.

- This study proposes a method for automatically generating a seed corpus and fuzzing executables by analyzing the test code in a library project to improve library fuzzing efficiency and minimize the interventions of the human expert.
- The source code [13] of the *FuzzBuilderEx* system proposed in this study has been published on the GitHub system to facilitate other researchers' reproduction of our results and use them for library fuzzing automation-related research.
- The effect of *FuzzBuilderEx* on code coverage increased by 31.2% and by 58.7% on a unique crash count, compared to that obtained via previous studies. This was achieved by applying nine open-source libraries to evaluate the performance and effectiveness of *FuzzBuilderEx*, which detected five vulnerabilities. Among them, three vulnerabilities are zero-day vulnerabilities. Notably, one vulnerability was registered in the common vulnerability and exposure (CVE).

II. RELATED WORK

We researched how to generate a seed corpus that is the initial input data for the base fuzzer and a fuzzing executable, which is the fuzzing target. Thus, we first discuss related work on the black-box and grey-box fuzzer. We then address related research on the automation of a seed corpus and fuzzing executable.

A. BLACK-BOX AND GREY-BOX FUZZING

Fuzzing is a dynamic test used to discover vulnerabilities in a program using random input values. Based on the information granularity observed during the fuzzing, this research divided the fuzzing into two types: black-box and grey-box fuzzing. We used the term grey-box instead of white-box in the software engineering field because fuzzing typically uses only limited information, unlike white-box testing.

The term black box is not to look inside the target program. It is named because it is like a black box to proceed by observing only the input and output of the target program. Many fuzzers such as SPIKE [14], BFF [15], FOE [16], and zzuf [17] adopted this approach since Miller [1] first introduced black-box fuzzing. Some modern fuzzers, such as funfuzz [18] and Peach [19], used this method, but the most recent research is being conducted based on grey-box fuzzing rather than black-box fuzzing.

Grey-box fuzzing generates random values efficiently to determine program paths using lightweight program analysis. Various grey-box fuzzers exist, but AFL [2], which is a coverage-based grey-box fuzzer, has been widely used, and considerable research has been conducted on AFL. Figure 1 illustrates the process of grey-box fuzzing.

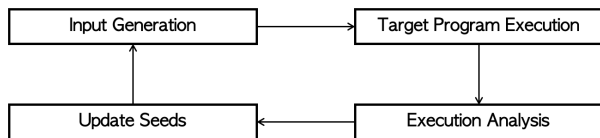


FIGURE 1. Process overview of grey-box fuzzing.

A grey-box fuzzer generates random values based on the provided seed and analyzes the execution information, such as the basic block coverage, during program execution. If the generated input value reaches new coverage, the grey-box fuzzer updates it in the seed queue and repeats this step continuously. However, the grey-box fuzzer generates random values based on the seed. Thus, it is challenging to achieve high coverage if the program requires a complex input value structure.

Many types of research to determine various program paths in fuzzing have been performed to address this problem. Researchers have suggested a hybrid approach, including VUzzer [20], T-Fuzz [5], QSYM [6], and Angora [7], which use symbolic execution or taint analysis as an input data generation mechanism to solve the problem. This research has led to a more efficient generation of fuzzing seeds, but they require additional runtime resources to apply symbolic execution or taint analysis. Unlike the above seed generation mechanism, the seed generation method proposed in this study is still a useful method because it helps reach sufficient code coverage without additional runtime resources.

B. AUTOMATED SEED CORPUS GENERATION

Research related to the generation of a seed corpus can be divided into sequence analysis and deep learning approaches.

We discuss the related research on each approach in the following subsections.

1) SEQUENCE ANALYSIS APPROACH

The seed corpus indicates the initial input data for the fuzzer. Previous research has analyzed the scenario of a fuzzing target program used to automatically generate these seed corpora and fuzz drivers. First, research to analyze the usage scenario analyzed the source code sequence using mining or code snippets. MAPO [21] was used to analyze the most frequent code sequences in code search engines. MLUP [22] expanded MAPO to determine the most widely used code sequence using a multilevel analysis of the control sequence. PAM [23] improved the ability to determine the code sequences using machine learning. Buse et al. [24] generated code sequences based on control flow. Finally, APIMiner [25], MUSE [26], and CLAMS [27] analyzed sequences using the program slicing technique.

In addition, research in the software engineering field has been conducted to automatically generate test cases related to a seed corpus. Randoop [28] generated sequences using feedback information as a guideline. RecGen [29] inferred a sequence in the sample code using each static and dynamic access method and generated a random test from the inferred sequence. Palus [30] improved the above tool by combining static and dynamic access methods. Pradel [31] generated a finite state machine (FSM) through dynamic analysis and generated an API call sequence based on the FSM.

The above research has made it possible to use the results of program sequence analysis in various fields. However, because the above studies analyze program sequences based on the target program, they have limitations in analyzing various usage scenarios and require high resources. In contrast, this study generates a seed corpus using test code function sequences instead of the target program; furthermore, it can generate an efficient seed corpus without additional effort because it uses the test code written in the development process. In general, this research differs from existing studies in that a seed can be automatically generated based on the primary usage scenario of the fuzzing target library.

2) DEEP LEARNING APPROACH

The above approach, based on sequence analysis, lacks scalability because it requires considerable specialized knowledge or substantial analysis resources. Thus, deep learning-based research was conducted to address this problem using the deep learning technique for seed generation. SmartSeed [32] executed the initial seed file using AFL and collected seeds that caused a unique crash or unique path as the training dataset. It then suggested a similar seed corpus generation method using a generative adversarial network model [33].

The related work using the deep learning approach generates another seed corpus using the existing seed corpus. This research has a different access method in that it generates a seed corpus based on the test code. Still, the seed corpus automatically generated by this research has limitations in

improving the fuzzing efficiency if the library test code does not have sufficient test quality or lacks test code. We intend to proceed with the research by applying the deep learning approach to the existing research as future work to solve this problem.

C. AUTOMATED FUZZING EXECUTABLE GENERATION

FUDGE [10], an automated fuzz driver generation system, makes it easy for developers to apply fuzzing to open-source libraries. FUDGE scans Google Codebase and extracts code snippets related to various target libraries. FuzzGen [11] is another recent research on generating a fuzz driver by recognizing the API. Similar to FUDGE, FuzzGen discovers the API sequence of the target library through a system analysis using the target library and compiles it into the LLVM bytecode. FuzzGen generates an abstraction API dependence graph (A2DG) that abstracts API interactions for a given library. FuzzGen generates an executable Libfuzzer [8] stub that executes an API sequence similar to a consumer program using A2DG.

Instead of extracting the code snippet from the consumer program that uses the target library, *FuzzBuilderEx* inserts instrumentation for all test codes in the target library. It also prioritizes a specific command that calls the defined FA to create an optimized executable file for more efficient fuzzing. In other words, the difference between this study and existing studies lies in the fact that it is generated based on the test code executed in the development process without analyzing a consumer program to create a fuzzing executable. *FuzzBuilderEx* is a lighter and more independent system compared with existing related systems, such as FuzzGen and FUDGE, because it only requires the library test code without having to analyze consumer programs.

III. MOTIVATION: LIMITATIONS OF PREVIOUS STUDY

FuzzBuilder [12], a previous study in this paper, is one of the State-of-the-Art (SOTA) studies for library fuzzing automation. Compared with SOTA research for similar purposes, such as FuzzGen [11] and FUDGE [10], discussed in Section II-C, FuzzBuilder increased the level of library fuzzing automation by automatically generating fuzzing seed corpus as well as fuzzing executables. Therefore, this study aims to maximize the automation level and efficiency by conducting additional research based on FuzzBuilder.

Previous research on FuzzBuilder contributed to reducing human experts' intervention for library fuzzing by providing a library fuzzing environment using a test framework. However, the previous research was limited in that the preliminary preparation for library fuzzing still required human expert intervention, and it could only be applied to limited functions. From an automation perspective, FuzzBuilder must manually create configuration files to automatically generate a seed corpus and fuzzing executables. Moreover, there is a limitation in that it should analyze APIs existing in the project to select the FA to be a fuzzing target. That is, FuzzBuilder still needs expert resources. According to Jiang *et al.* [34],

inferring FAs by human effort is a challenging job. Therefore, this study automatically selects APIs that can be candidates for FA and generate FA configuration files. This allows the fuzzing process, which is partially automated, to be expanded to the entire process.

We can consider fuzzing efficiency from two aspects: execution speed and coverage. First, FuzzBuilder used a limited method of removing only test functions that did not include FA in terms of execution speed. We confirmed that the execution speed was approximately 18% slower than that of the manually created fuzzing executables. In this study, not only are functions that do not include FA eliminated but also unnecessary syntax within functions are excluded and several fuzzing executables are generated by a logical unit that contains FA, thereby improving the fuzzing speed. Second, FuzzBuilder had limitations in performing fuzzing on various functions from the perspective of fuzzing coverage because only functions with string-type parameters were available as FA. According to Fan *et al.* [35], integer overflow is one of the top ten vulnerabilities reported over the 20 years. Moreover, we confirmed through nine test projects that we are able to infer about 27% more FAs by expanding Integer Type. Therefore, this study improves fuzzing coverage by extending the target FA as functions with an integer-type parameter.

IV. PRELIMINARIES

This section describes the background knowledge required to understand this study. First, Section IV-A discusses what library fuzzing is and why library fuzzing is necessary and difficult. Section IV-B describes the test framework used in this study to automate the generation of fuzzing executables and seeds and address why it is helpful to utilize it. Section IV-C explains why this study uses fuzzing coverage to measure fuzzing efficiency. Finally, Section IV-D defines the assumption and scope of this research.

A. LIBRARY FUZZING

A library is a collection of functions that provide a defined interface. Various applications can use the features provided by libraries. This reuse of a library provides developers with high productivity, contributing significantly to software development. However, a library's vulnerability can be more critical and fatal than that of other applications because various programs can be affected by vulnerable libraries. Therefore, it is essential to test using fuzzing or other methods to preemptively prevent a library's vulnerability.

The application uses library functionality by invoking the library API function. A library also operates based on a set of functional sequences with valid parameters. Therefore, library fuzzing requires a fuzz driver that passes parameter values to the library and executes the library function sequence. When constructing a fuzz driver, the target API function for fuzzing is identified, and the API call sequence must be properly performed using valid parameters. To construct a fuzz driver, application developers or testers must understand the target library's design and how the fuzzer

engine works. As such, the application developer's use of valid parameters for fuzzing performance and the creation of a fully buildable fuzz driver that executes the sequence of the target API functions can serve as overhead during development.

Libfuzzer [8] is a grey-box fuzzer that targets the most used library. The *LLVMFuzzerTestOneInput* function, which developers use as the interface between the fuzzer and target library, must be defined for Libfuzzer. This function is called a fuzz target function. The fuzz target function has two parameters. The first is the buffer address that stores the value passed by the fuzzer, and the second is the size of the buffer.

Thus far, OSSFuzz [9], which is Google's continuous fuzzing service for open-source software, tested more than 100 open-source projects and found more than 10,000 vulnerabilities. To register an open-source project with OSSFuzz, the project developer must manually create a fuzz driver. OSSFuzz stores the source code and deploys the distributed environment in which fuzzing is performed in ClusterFuzz. OSSFuzz used a Libfuzzer as a base fuzzer. As previously described, the Libfuzzer is a grey-box fuzzer for a library, which requires developers to create a fuzz driver using the fuzz target function (*LLVMFuzzerTestOneInput*) and generate an executable file using Libfuzzer.

Although Libfuzzer provides a library fuzzing method, it requires a human expert's aid; Thus, an expert with a high understanding of the target project must manually write fuzz drivers. In the course of doing so, determining interesting API calls, API dependencies, and fuzzed arguments is at the sole discretion of the human expert. That is why using Libfuzzer as a fuzz driver is challenging because it requires a deep understanding of the target library, a fuzzer engine for fuzzing friendly code writing, and continuous maintenance of the fuzz driver code. Therefore, this approach cannot be applied to various libraries in general.

B. TEST FRAMEWORK

Testing must be performed during the development process to improve the reliability and quality of software programs. In the software, a unit indicates a group of subprograms or subprograms. Analyzing and validating all codes requires substantial effort if the software has a complex structure. Testing in units, rather than overall, can be an excellent way to solve these problems.

The test framework based on xUnit [36] was first developed as a Smalltalk version by Kent Beck. Based on xUnit, the framework is used in various languages, such as JUnit [37], NUnit [38], and cppUnit [39]. Recently, Google Test [40], developed by Google, has been widely used as a test framework for open-source projects. Testing using the test framework is an essential step in the software development process, as the agile methodology is widely applied to software development. The test framework provided the following functions:

First, tests should be independent and repetitive. Debugging test functions that succeed or fail as a result of other

test functions is quite difficult. Therefore, the test framework isolates the test by running the test on various objects and ensures independent execution so that the failure or success of various test functions does not affect a particular test function.

Second, the test framework groups related test codes that can share data and subroutines into test suites. These characteristics allow the generation of independent test codes with a variety of more complex test codes at higher levels.

Third, the test framework should allow developers to focus on testing. The test framework automatically tracks all defined tasks and does not require developers to perform additional tasks to execute the test code. Furthermore, developers can reuse shared resources between test codes and establish and release shared resources independently without dependence between test codes. Testing, based on a test framework that provides these features, allows developers to verify that the program functions correctly at the development stage.

For successful testing, the test must contain all the possible program paths. Thus, the test codes in the development process also require high code coverage. We must also consider the function sequence to explore various program paths and use various test input data as parameters to call specific functions. Therefore, various function sequences and test input data can be used in the test code if the test code is well written during the development process. Because the method proposed in this work uses the test framework, we cannot apply it without the test code of the library project. However, this approach is still valid because analyzing the top 112 most star libraries registered on GitHub [41] and OSSFuzz [9] showed that 103 libraries (92%) had a test framework.

C. CODE COVERAGE IN THE FUZZING

The grey-box fuzzer establishes a fuzzing strategy based on the assumption that high code coverage is more likely to detect vulnerabilities [42]. In particular, to improve code coverage during library fuzzing, it is necessary to understand the function sequence and select an efficient seed.

The writing code for library fuzzing begins by selecting a base function. In addition, other functions related to the base function must be called to achieve sufficient code coverage. If the function requires a return value from another function, we cannot test the library's functions alone. In such cases, we jointly tested all the relevant functions. Therefore, the code for library fuzzing should be written with a set of function calls that include the relevant functions. This paper defines the base function selected for fuzzing as the FA and defines the set of function calls of all functions associated with the base function as a function sequence for testing.

Low test coverage is a typical weakness of grey-box fuzzing [42]. Because grey-box fuzzing relies on randomness, it is challenging to navigate deep program paths [5]. Therefore, to mitigate these shortcomings, efficient seeds should be provided to the grey-box fuzzer [43]. The seed must explore a variety of deep program paths without the input generation mechanism of grey-box fuzzing. Choosing the appropriate

seed can help increase test coverage by exploring more program paths. Rather than starting from scratch without a seed corpus or using random seed values, it is better to generate and provide an efficient seed. This work includes a study to generate efficient seeds by automatically employing test codes.

As discussed in Section IV-B, the test code implemented during the development process contains deep levels of information because developers (experts) write it with a high understanding of the library. However, including deep levels of information does not necessarily mean high coverage; further improvements in coverage require the explicit addition of test codes. In software engineering, it is generally considered impossible to test many subpaths, even if tested by invoking all APIs through the test code [44]. However, the fuzz engine transforms seed files using a mutation strategy and produces various input data without explicit intent. Thus, in this study, test data generated using these high-quality test codes can be expected to achieve higher code coverage by using them as seed files instead of directly using them as inputs to fuzzing.

D. ASSUMPTION AND SCOPE

We assume that the target library has a test code that applies the test framework because the system in this study operates based on the test framework. This assumption is valid because the results of analyzing the top 112 most star libraries registered on GitHub and OSSFuzz showed that 103 libraries (92%) had a test framework. Each test function must be independently configured and executed to automatically generate a fuzzing executable based on the test code. Well-known test frameworks, such as Google Test and xUnit, which were previously introduced in Section IV-B, follow the above conditions, and many projects have independently written test functions. Thus, the overall conditions do not have a significant impact on the practical aspects of this approach.

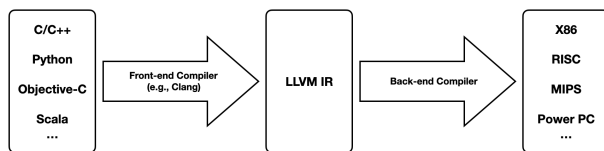


FIGURE 2. Overview of *LLVM IR*. LLVM is designed around a language independent intermediate representation that serves as a portable, high-level assembly language that can be optimized with a variety of transformations over multiple passes.

This study focuses on the generation of a fuzzing executable and seed corpus to provide an automated environment for library fuzzing. Thus, the fuzzer itself, such as the fuzzer algorithm, is not the scope of this study, and this study configured the system to be compatible with various grey-box fuzzers as well as AFL. The C/C++ code is a primary target for fuzzing because of unsafe language features, such as explicit memory management, which makes it prone to bugs and vulnerabilities. In addition, we selected C/C++ as the target language because it is a major language for open-source libraries. Furthermore, we converted the source code into an

LLVM bitcode for scalability and performed instrumentation using the LLVM API [45]. As shown in Figure 2, the LLVM can be converted into the same type of intermediate representation (IR) using each compiler for different languages such as C, Python, and Objective-C. In other words, because multiple languages can transform the same LLVM IR form, we insert instrumentation code in the form of LLVM bitcode rather than directly source code into the target language, considering the scalability of further studies.

Fuzzing can be classified into one executed by the developer and by the tester. In the developer's case, it is easy to select FAs and write a fuzzing code, but it may be challenging to apply the target library fuzzing. However, in the case of the tester, it is easy to apply the fuzzer and perform fuzzing, but it may be challenging to select FAs and generate an efficient seed corpus and fuzzing executable because of the lack of the target library's understanding. This study focuses on fuzzing by the tester. Because this study assumed that testers employed the test framework of the target program, they can access its source code and non-obfuscated binary.

V. MAIN METHOD

This section describes *FuzzBuilderEx* proposed in this study. Figure 3 shows the execution order of the *FuzzBuilderEx*. First, in the Inferring FA phase, FA is automatically selected by using the library and test binary, and in the Generating Seed Corpus phase, the seed corpus is generated using the test code and FA. Subsequently, in the Connecting fuzzy and library phases, the fuzzer replaces the FA parameters to feed the seed corpus to the fuzzing target. Finally, in the synthesizing fuzzing executable phase, a fuzzing executable file is created using the test framework. At the beginning of each subsection, major considerations in the *FuzzBuilderEx* design are addressed as challenging points. The details of each phase are described in the following subsections.

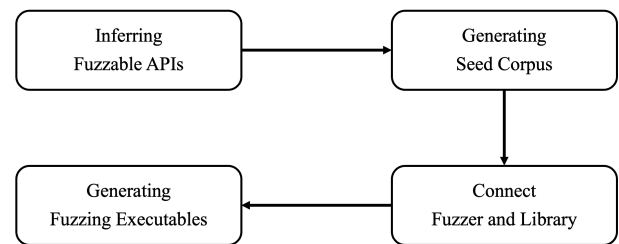


FIGURE 3. Process overview of *FuzzBuilderEx*. It has four phases to provide an automated library fuzzing environment.

A. DESIGN CONSIDERATION

Before describing the detailed design of *FuzzBuilderEx*, we discuss the design consideration.

First, even if the source code of the target program is continuously updated, it should not be affected, and *FuzzBuilderEx* should consider implementation language scalability. Because target programs are constantly updated, such as functional improvements and bug fixes, designing *FuzzBuilderEx* based on a specific source code may make it

impossible to apply them during the target program updates. In addition, we should consider scalability for application to libraries implemented in other languages, as well as C/C++. Therefore, we designed to apply *FuzzBuilderEx* by converting it to the intermediate language form, LLVM bytecode.

Second, *FuzzBuilderEx* should automate the entire process of preparing for library fuzzing to minimize expert intervention. Thus, this research automated the entire process for preparing the library fuzzing environment, including configuration information, seed corpus, and fuzzing executables generation.

Third, *FuzzBuilderEx* should improve fuzzing efficiency. This study employs a test framework to generate a seed corpus and fuzzing executables. We generate seed corpora with extension FA's parameter to improve fuzzing coverage, and remove unnecessary functions and code statements for fuzzing executable to improve fuzzing speed.

B. INFERRING FUZZABLE APIS

Challenging Point 1: How can we automatically infer FA without expert intervention? This study extracts common functions using target binary and test binary, and automatically extracts FA using information obtained through source code parsing. Note that we can automatically infer FAs using test binary, and target sources without expert intervention.

Challenging Point 2: How can a configuration file be created for enhancing the automation level? Configuration files, including FA and parameter information, are required to generate a seed corpus and fuzzing executables automatically, but the previous study created configuration files manually. Thus, this study created configuration files automatically using the FA and parameter information extracted earlier.

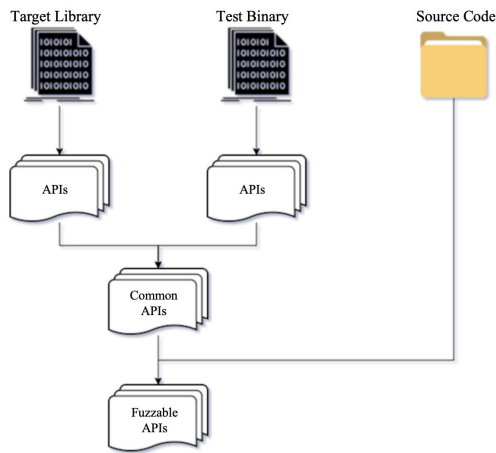


FIGURE 4. Process overview of inferring FA. Common API is generated using target library and test binary, and FAs are selected considering FA's constraints.

FA selection for library fuzzing requires many testers' interventions, such as library analysis. Thus, this paper proposes an automated method of inferring FAs, as shown in Figure 4, by imitating the method and process used to select

an FA manually by a tester.

$$\mathcal{F}_{target} \cap \mathcal{F}_{test} \rightarrow \mathcal{F}_{common} \quad (1)$$

Notably, when test code is insufficient, it may not contain some APIs of the target program; thus, we should conduct Equation (1) to reduce unnecessary operation. This study selects FA based on the test case, so it is an API for the target library, but some APIs are not used in the test case. Thus, to filter out unnecessary APIs, we identify the APIs commonly used by the target program's API and the test program. Equation (1) describes the process of extracting the common functions: All functions list ($\mathcal{F}_{target}$) of the target library is first extracted, and the callee function list ($\mathcal{F}_{test}$) is also extracted from the test binary, which validates the target library. Subsequently, a common API list ($\mathcal{F}_{common}$) was collected from each extracted list.

$$Filter_s(\mathcal{F}_{common}) \rightarrow \mathcal{F}_{FA} \quad (2)$$

Algorithm 1 Inferring FAs

```

1: procedure Get_FA_List(library, test_bin, lib_src)
2:   target_funcs ← get_target_funcs(library)
3:   test_funcs ← get_test_funcs(test_bin)
4:
5:   for func ← target_funcs do
6:     if find_func_in_test(func) then
7:       add_common_func_list(func)
8:     end if
9:   end for
10:
11:   common_funcs ← get_common_funcs()
12:
13:   for func ← common_funcs do
14:     if has_string_or_integer_params(func) then
15:       add_fa_list(func)
16:     end if
17:   end for
18:
19:   fa_list ← get_fa_list()
20:
21:   return fa_list
22: end procedure
  
```

Equation (2) describes the process for selecting the FA list by applying the *filter* to the common API. As addressed in Section IV-A, the FA must receive the seed transmitted from the fuzzer and use it as a parameter, and this study has limited the FA to functions with a string or integer-type parameter. Thus, the filter ($filter_s$) finally selects the FA through the prototype analysis of the source code's function. That is, the function's prototype is identified from the source code (*s*) and further selected as the FA by checking the validity, such as by inspecting whether the type of parameter in each API.

Through the inferring FA phase, we can auto-generate the seed configuration file, which includes FAs and parameter information, as shown in Listing 1. In a nutshell, *FuzzBuilderEx* automatically generates a seed configuration file for each FA by using the auto-inferred FA list and can create a seed corpus for fuzzing by using this configuration file (refer to Section V-C). Algorithm 1 shows the detailed process of inferring the FA phase.

C. GENERATING SEED CORPUS

Challenging Point 1: How can a valuable seed file be generated without the analysis of input data? The test data are input data for verifying the core path by a developer; thus, we generated a seed corpus using the test data used as parameters for FA in the test framework.

Challenging Point 2: When should the test data be captured to generate a seed file? The test data used as input data of the library have different values depending on the capture point because changes occur during the running of the library. A capture of the values entered into the FA during the runtime of the library was implemented. In this case, the library automatically changes to fit the data format without analyzing the input data of the FA so that it can be used as a seed file without additional analysis.

The previously generated seed configuration file contains parameter information on the FA and can generate the seed corpus using the following four types: The first type specifies only the index of the string-type parameter. The second type is a format that specifies both string type and integer-type parameters, that is, the length information of the string type. In addition, to use the integer type more widely, we added the third type that specifies one more integer-type parameter to the second type and the fourth type that selects only the integer-type parameter. Listing 1 below shows examples of the four types of seed configuration files.

```

1 #XML_Parse
2 # (XML_Parse parser, const char *s, int len, int
   isFinal)
3
4 #Seed Configuration Type (1) {
5   "targets": [[ "XML_Parse", 2 ]],
6   "files": [ "xmlparse.bc" ]
7 }
8 #Seed Configuration Type (2) {
9   "targets": [[ "XML_Parse", 2, 3 ]],
10  "files": [ "xmlparse.bc" ]
11 }
12 #Seed Configuration Type (3) {
13   "targets": [[ "XML_Parse", 2, 3, 3 ]],
14   "files": [ "xmlparse.bc" ]
15 }
16 #Seed Configuration Type (4) {
17   "targets": [[ "XML_Parse", 0, 0, 4 ]],
18   "files": [ "xmlparse.bc" ]
19 }
```

Listing 1. Types of seed configuration.

Listing 2 shows the limitations when fuzzing is performed using only the string-type parameter of the FA. The *vuln_01* function does not check the input value's length before calling the *strcpy*. Consequently, buffer overflow vulnerability can

occur. However, because the condition that the flag must be 1 is required, the input value cannot reach the vulnerable code statement when fuzzing with the previous study's system, which can use only a string-type FA parameter. In addition, the function *vul_02* may cause buffer overflow through the *str_len* parameters. The *vul_03*, *vul_04* may cause an integer overflow, one of the significant CVE [46] in the C/C++ language. However, there is a limitation that it is challenging to induce such a crash because it does not include an integer-type FA for existing studies. Thus, the addition of an integer type can achieve higher coverage than that of the previous study and achieve high efficiency. Assuming that the FA is the *XML_Parse* of line 2 in listing 1, we can configure the configuration file for seed generation in four ways. In the target configuration, the first value designates the FA name, and the second, third, and fourth values specify the position of the FA string argument, string length argument, and an integer argument. If the test code receives the string length value as input, it can be used, as shown in line 16. If the FA contains only integer arguments, the second and third values are zero, as shown in line 21. The fourth value is required to specify the integer argument location of the FA.

```

1 /* MAY NOT test buffer overflow of strcpy */
2 void vuln_01(char* str, int flag) {
3     char buffer[24];
4
5     if (flag == 1)
6         strcpy(buffer, str);
7 }
8
9 /* MAY NOT test buffer overflow of strncpy */
10 void vul_02(char *str, int str_len) {
11     char buffer[24];
12     strncpy(buffer, str, str_len);
13 }
14
15 /* Do NOT test integer overflow */
16 void vul_03(int num) {
17     unsigned short s;
18     s = num;
19 }
20
21 /* Do NOT test integer overflow */
22 void vul_04(char *arr, int index) {
23     int value;
24     value = arr[index];
25 }
```

Listing 2. Limitation of previous study.

The test input data to generate a seed corpus was used in this research. In other words, the seed corpus can be extracted from the test input data used as a parameter of an FA. Typically, a software project stores the test input data in the source code or a separate file for testing. As addressed in Section II-B1, we can extract the test input data into the sequence information using various static analyzers. However, static analysis tools that analyze source code without execution information have many false positives. If this is used to create a seed file, unnecessary fuzzing runs increase, and efficiency decreases. It is not practical to prepare various static analyzers. Thus, this study utilized a dynamic approach to collect test input data while testing was performed. For this purpose, *FuzzBuilderEx* instruments the LLVM IR code in the

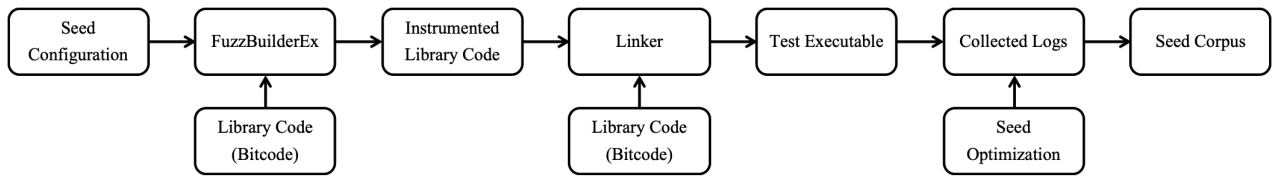


FIGURE 5. Overall process to generate seed corpus. First, the modified library bytecode and test code are used to collect logs, and then, the seed corpus is generated through parsing and optimizing collected logs.

library FA and stores the value passed to the parameter in a specific file. Listing 3 shows how instrumentation is used for seed corpus collection. After finding the API selected as the FA in the library bytecode, *FuzzBuilderEx* stores all the values entered as parameters through LLVM IR instrumentation in a specific file. Subsequently, *FuzzBuilderEx* creates a seed corpus by separating the seeds stored in a specific file into a file unit. We describe the detailed algorithm for generating the seed corpus at the LLVM IR level in Appendix B.

Because the seed corpus automatically generated through the preceding process can induce redundant fuzzing, it is necessary to select valid seeds by applying seed optimization. In other words, using a seed file with a large size reduces the performance of the fuzzer compared to a seed file with a small size that has the same coverage. The AFL provides a cmin tool to address this problem by removing redundant seed files based on the dry run of the AFL before the fuzzer run. The seed optimization tool, such as AFL-cmin, allows for increased fuzzing efficiency by creating a minimal set of seed files that do not overlap. Thus, this sample test optimized the seed corpus using AFL-cmin, which supports corpus minimization of AFL. Figure 5 shows the overall process of the Generating Seed Corpus stage.

```

1 void FA(const char *str, int len, int flag) {
2     // to collect seed corpus
3     ofstream log ("seed.collect");
4     log << str;
5     log.close();
6     ...
7 }

```

Listing 3. The example of instrumentation for seed corpus.

D. CONNECTING FUZZER AND LIBRARY

Challenging Point 1: How can the seed file be fed to the library from the fuzzer? To connect the fuzzer and target library, the fuzzer must provide seed files to the target library. This research links the fuzzer and the library by storing seed files in global variables and replacing the FA's parameters with global variables containing seed information.

The fuzzer and fuzzing target library must be connected to transfer and use the seed corpus provided by the fuzzer to the fuzzing target library. In other words, the fuzzer must read the value from the seed and feed it to the FA parameter. For this purpose, *FuzzBuilderEx* extracts the entry function (main function) from the test binary. To extract the entry function,

FuzzBuilderEx searches the entry function in every function of the LLVM bytecode file.

After reading the input value from the fuzzer using stdin or a specific file, *FuzzBuilderEx* inserts the interface, which loads the corresponding input value in the memory into the entry function., three new global variables are added to store both string and integer types. These global variables include two variables that store the value read from each seed and the corresponding size of the value. Another global variable has this value in the case of an FA that uses an integer as a parameter. Subsequently, *FuzzBuilderEx* instruments a sequence of instructions into the extracted entry function to obtain the input value from the fuzzer and allocate sufficient space for the added global variables. Then, *FuzzBuilderEx* copies the input value into the added global variables, and if there is no value depending on the parameter type, a null value is entered. As previously described, *FuzzBuilderEx* uses a method that uses stdin or a specific file to read input values from the fuzzer. Because most grey-box fuzzers use stdin or a specific file as their interface, this research can guarantee compatibility with automatically generated fuzzing executables and various grey-box fuzzers (such as AFL).

```

1 void TEST_1() {
2     /* init for API_1 */
3     API_1(parser, test_input, strlen(test_input), opts)
4     ;
5     /* check result of API_1 */
6 }
7 void TEST2() {
8     /* init for API_2 */
9     API_2(parser, test_input, strlen(test_input), flag)
10    ;
11    /* check result of API_2 */
12 }
13 int main(int argc, char char **argv) {
14     TEST_1();
15     TEST_2();
16
17     return 0;
18 }

```

Listing 4. The example before replacing FA parameter.

FuzzBuilderEx changes the FA parameter with previously added global variables. The information on the parameters to be changed is identified using the FA parameter information written in the previously created configuration file. Through this process, before the FA is called, *FuzzBuilderEx* reads the value from the fuzzer and stores it into a global variable, and the FA can perform fuzzing by using this value as a parameter.

```

1 char *FUZZ_INPUT;
2 int FUZZ_INPUT_LEN;
3
4 void TEST_1() {
5     /* init for API_1 */
6     result = API_1(parser, FUZZ_INPUT, FUZZ_INPUT_LEN,
7         opts);
8     /* check result of API_1 */
9     EXPECT_EQ(True, result)
10 }
11
12 void TEST2() {
13     /* init for API_2 */
14     result = API_2(parser, FUZZ_INPUT, FUZZ_INPUT_LEN,
15         flag);
16     /* check result of API_2 */
17     EXPECT_EQ(True, result)
18 }
19
20 int main(int argc, char char **argv) {
21     read_from_fuzzer(&FUZZ_INPUT, &FUZZ_INPUT_LEN);
22
23     TEST_1();
24     TEST_2();
25
26     return 0;
27 }

```

Listing 5. The example after replacing FA parameter.

Listings 4 and 5 describe example codes for replacing the FA parameter values. Listing 4 is the code in the state before the FA parameter is substituted, and is an example of the test code of the library project. Listing 5 is an example of the changed code so that the parameters of the FA are replaced, and the seed can be fetched from the fuzzer. Figure 6 shows the overall process of replacing the FA parameters.

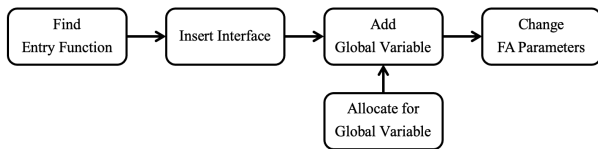


FIGURE 6. Overall process to connect fuzzer and library. Replacing FA parameters. Replacing FA parameters is performed to feed a seed corpus provided by fuzzer to the target library.

E. GENERATING FUZZING EXECUTABLES

Challenging Point 1: How can we identify the call sequence between FAs? The test code implements the correlation of the call sequences between the FAs. We consider these relationships to create fuzzing executables.

Challenging Point 2: How can the slowdown of fuzzing execution be mitigated? This study utilizes test codes to generate fuzzing executables, so these contain unnecessary functions and code statements. Thus, we remove test functions that do not include FAs and delete code statements that do not correlate with fuzzing within the test function. In addition, we create several fuzzing executables by dividing the executable into several functional units to maximize parallelism.

The test framework creates the program to be tested as an independent executable file and performs the test. This study converted the generated executable file created by the test framework into a fuzzing executable file. In the Connecting

Fuzzer and Library phase, we identified the entry function, read the input value from the fuzzer into the identified entry function, and completed the code insertion to deliver the value to the fuzzing target library. Subsequently, the process for generating the fuzzing executable consists of (1) the identity test function step, (2) the optimized executable step, and (3) the Synthesized fuzzy executable step. The details of each step are presented in Figure 7. We describe the detailed algorithm for generating fuzzing executables at the LLVM IR level in Appendix C.

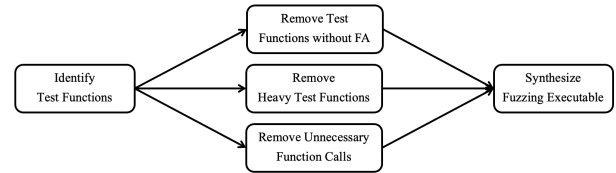


FIGURE 7. Overall process to generating fuzzing executable. The fuzzing executable is created after deleting unnecessary code statements for fuzzing in test functions.

1) IDENTIFY TEST FUNCTIONS

A test function is a logical module that implements the test case to be analyzed using *FuzzBuilderEx*. Although there are many functions of the test code in the library project, only some of them are related to the test functions. Thus, *FuzzBuilderEx* needs to identify the functions that are test functions. If a test is based on a widely used test framework such as GoogleTest, there is a specific pattern in the test function names. Thus, it is not difficult to identify the test function in this case. However, suppose testing is based on the in-house test framework, and the test function's name is created without any pattern. In this case, because there is no clear way to determine if that function is a test function, the name is manually specified using a configuration file. However, according to the analysis of the OSSFuzz projects, most of the projects use test function names with specific prefixes or suffixes, such as *test_functionA*, *test_functionB*, *functionA_test*, and *functionB_test*. Thus, *FuzzBuilderEx* supports the asterisk (*) and can identify the test function when using the in-house test framework by writing rules, such as the *test_** and **_test*, in the configuration file.

2) OPTIMIZE FUZZING EXECUTABLES

The automatically generated fuzzing executable based on the test code contains function calls or operations that are unrelated to fuzzing. For this purpose, this study added a step to optimize the fuzzing executable. Because the previous study limited the application of the optimization process, the test function that did not contain the FA or code statements to check the test result was repeatedly run in the previous study's fuzzing executable. Accordingly, it is possible to make the fuzzing executable lighter and more efficient by removing unnecessary functions by understanding the calling relationship between functions and whether fuzzing is related. Optimization through elimination is possible because

the test functions can be executed independently, and each test function does not affect one another. The details of each optimization method are described below.

- **Remove test functions without FA:** Identify the test function that contains the statement calling the FA selected from among the previously identified test functions. If there is a statement that calls the FA, the test function is preserved; otherwise, the test function is deleted.

- **Remove heavy test functions:** Even in the case of a test function including the FA, if it is not useful for fuzzing for whatever reason, the user can explicitly delete that function. For example, in some test functions, if there are many operations, such as the excessive use of loop statements and memory read/write, the execution speed becomes slower, and if the execution speed is slow, the efficiency of fuzzing decreases. In this case, by explicitly designating these heavy functions in the configuration file, the corresponding test function can be removed from the fuzzing executable.

- **Remove unnecessary function call:** In the test code, operations that simply check or compare test statuses or results are frequently used. For example, an operation such as *EXPECT_EQ*, which compares the expected result and the test result, is iteratively used. While it is not a statement that affects fuzzing, this code is an essential statement for checking the test results. Thus, this study explicitly designated this unnecessary function call in the configuration file to remove this code from the fuzzing executable. However, if there is another function call in the parameter, it can affect fuzzing. Thus, this study removed only unnecessary function calls and preserved the parameters. In this case, although meaningless code, such as code in which only simple variables corresponding to the parameters are declared, may remain, this can be resolved by using the optimization option when compiling bytecode.

```

1 char *FUZZ_INPUT;
2 int FUZZ_INPUT_LEN;
3
4 void TEST_1() {
5     /* init for API_1 */
6     result = API_1(parser, FUZZ_INPUT, FUZZ_INPUT_LEN,
7         opts);
8     /* check result of API_1 */
9     EXPECT_EQ(True, result)
10 }
11
12 void TEST2() {
13     /* init for API_2 */
14     result = API_2(parser, FUZZ_INPUT, FUZZ_INPUT_LEN,
15         flag));
16     /* check result of API_2 */
17     EXPECT_EQ(True, result)
18 }
19
20 int main(int argc, char char **argv) {
21     read_from_fuzzer(&FUZZ_INPUT, &FUZZ_INPUT_LEN);
22     TEST_1();
23     TEST_2();
24     return 0;
25 }
```

Listing 6. The example before optimizing executable.

Listings 6 and 7 describe an example code that applies executable optimization. Listing 6 is the code before the

```

1 char *FUZZ_INPUT;
2 int FUZZ_INPUT_LEN;
3
4 void TEST2() {
5     /* init for API_2 */
6     API_2(parser, FUZZ_INPUT, FUZZ_INPUT_LEN, flag));
7     /* check result of API_2 */
8 }
9
10 int main(int argc, char char **argv) {
11     read_from_fuzzer(&FUZZ_INPUT, &FUZZ_INPUT_LEN);
12     TEST_2();
13     return 0;
14 }
```

Listing 7. The example after optimizing executable.

optimization is used and includes test functions that do not contain FA and unnecessary function calls in a test function. Listing 7 is the code after optimization is applied, and test functions that do not include FA and unnecessary function calls have been deleted. Notably, the fuzzing executable's efficiency is expected to significantly increase by eliminating repeatedly used function calls such as *EXPECT_EQ*.

3) SYNTHESIZE FUZZING EXECUTABLE

The instrumented code generated through the preceding process was built into a fuzzing executable according to the library's build process. Therefore, there was an interface to input values from the fuzzer, and a fuzzing executable composed of optimized test functions were generated. Although the fuzzing executable can be configured in various ways, to maximize the fuzzing efficiency using the parallel fuzzing feature of AFL, generating the fuzzing executable using a minimal size and by the maximum number is recommended. Thus, this paper generated fuzzing executables per FA unit to satisfy this condition.

VI. EVALUATION

A. EVALUATION METRICS

We utilized the widely used evaluation metrics [47] for *FuzzBuilderEx*. The detailed metrics are as follows:

- **Code coverage:** Code coverage is the most common indicator for measuring fuzzing efficiency, and is the number of touched code statements of the target program.
- **Crash count:** The crash count refers to the number of crashes in the target program during fuzz running and is the most commonly used fuzzing performance indicator with code coverage. In particular, the quality of FA relations within the fuzzing executable heavily influences the crash count. Therefore, this study used it to measure the effectiveness of fuzzing executables.
- **Execution speed:** Execution speed means execution command counts per second. The fuzzing speed itself does not imply fuzzing efficiency, but in general, a fast fuzzing speed means efficient fuzzing if it has the same coverage. We compared the execution speed of the fuzzing executable with related studies. However, because the execution speed can be affected by the

fuzzing running environment, such as system specifications, we used a relative value measured in the same environment.

- **Vulnerability:** We selected the target library to evaluate the capability of the vulnerability detection capabilities and then analyzed the vulnerability detection results.

B. EXPERIMENT SETUP AND TAKEAWAYS

We established experimental takeaways for our study to clarify research questions that validate our approach's effectiveness in achieving higher efficiency for library fuzzing. The experimental takeaway is shown below.

- RQ#1. How much does *FuzzBuilderEx* reduce human expert intervention? (refer to Section VI-C)
- RQ#2. How effective are an automatically generated seed corpus and executable files? (refer to Sections VI-D and VI-E)
- RQ#3. How practical is *FuzzBuilderEx* for detecting library vulnerabilities? (refer to Section VI-F)
- RQ#4. To what extent is *FuzzBuilderEx* more effective than previous research? (refer to Section VI-C-VI-F)

This study improved the limitations of prior research, FuzzBuilder, in automation, fuzzing execution speed, and fuzzing coverage. First, we enumerate the steps required to perform library fuzzing to verify improvement in the automation level and compare whether each step is automated. Second, as discussed in Section V-A, this study uses the test executable to generate a fuzzing executable, which may result in a slower execution speed compared to a manually generated fuzzing executable. In other words, a tradeoff can occur in which the level of automation increases and the speed of fuzzing execution decreases. To address this problem, we conducted research to minimize execution speed reduction. Therefore, we performed a relative comparison of the execution speed (see Figure 9) shown in the dashboard of the AFL to verify that we improved the execution speed compared to the preceding study. Third, as discussed in Section IV-C, fuzzing coverage is the most representative indicator of fuzzing efficiency [47]. To verify the effectiveness of this study in enhancing the code coverage, we compared the code coverage of the automatically generated seed and fuzzing executable in Section VI-D and Section VI-E, respectively. In addition, in the case of the fuzzing executable, we designed an experiment to compare the number of crash occurrences as well as the coverage to make a more holistic comparison. Finally, we evaluate the capability of vulnerability detection to demonstrate that the efficiency of fuzzing improves through increased fuzzing speed and coverage. In particular, we can comprehensively verify the effectiveness of *FuzzBuilderEx* through unknown vulnerability analysis as well as known vulnerabilities. Table 1 presents the experimental design.

We chose the most stars C/C++ libraries in GitHub and selected libraries that were applied to both OSSFuzz and FuzzBuilder. However, suppose no test code in the project or test case is related to the FA in the test code. In that case, it is

TABLE 1. Experimental design.

Problem Definition	Solution	Research Question
Automation	Automation of FA selection and configuration	RQ1, RQ4
Fuzzing speed	Remove unnecessary function and code statement Create fuzzing executable file by function unit	RQ2, RQ4
Fuzzing coverage	Add Integer type parameter for FA	RQ2, RQ3, RQ4

impossible to automatically generate a fuzzing environment for this study; therefore, this experiment excluded any such projects or text cases from the target library. In addition, for a fair comparison with OSSFuzz, an experiment was performed using the same FAs applied to each target library of OSSFuzz. Based on these criteria, this study applied *FuzzBuilderEx* to nine libraries. We applied OSSFuzz, FuzzBuilder, and *FuzzBuilderEx* to seven libraries for performance comparison. Additionally, we selected two libraries that were applied only to one FuzzBuilder or OSSFuzz. Table 2 lists the target library information used in the experiments.

TABLE 2. Fuzzing target library.

Target Library	Version (commit id)	OSSFuzz	FuzzBuilder
boringsl [48]	v3945 (d2a0ffd)	O	O
c-ares [49]	v1.15.0 (9c2068e)	O	O
expat [50]	v2.2.6 (39e487d)	O	O
http-parser [51]	v2.9.4 (ec8b5ee)	O	O
json-c [52]	v0.15 (df27756)	O	O
libhttp [53]	v0.5.35 (e198e8f)	O	O
yara [54]	v3.8.1 (a3784d3)	O	O
mpc [55]	v1.8.4 (b31e02e)	O	X
cJSON [56]	v1.7.8 (08103f0)	X	O

The line coverage and unique crash count which are the most commonly used metrics, were used to compare the fuzzing efficiency. We used AFL 2.52b with an address sanitizer [57]. Because AFL does not support an address sanitizer for 64-bit binaries, we built all target libraries as 32-bit.

C. COMPARISON OF LIBRARY FUZZING AUTOMATION

To evaluate the extent to which human expert intervention is reduced through *FuzzBuilderEx*, we listed library fuzzing procedures and compared the automation level by analyzing the human expert intervention for each process.

TABLE 3. Library fuzzing process.

Library Fuzzing Process	OSSFuzz	FuzzBuilder (the previous work)	FuzzBuilderEx (this research)
FA Selection	X	X	O
Seed Corpus Generation	X	O	O
Fuzzing Executable Generation	X	O	O
Fuzzing Test	O	X	X

Table 3 lists whether OSSFuzz, FuzzBuilder, and *FuzzBuilderEx* were automated for each step. OSSFuzz aims to perform the fuzzing test automatically, so it does not cover the fuzzing environment's preparation step. FuzzBuilder increased the automation level by automatically generating

the seed corpus and fuzzing executable based on the test framework. However, it still has the limitation that it must manually perform the FA selection. To solve this problem, in this study, *FuzzBuilderEx* significantly increased the automation level of library fuzzing by automatically selecting FAs and generating a seed corpus and fuzzing executable. As discussed in Section IV-D, it is not within the scope of this study to automate the fuzzing test itself. Notably, the automated fuzzing test of OSSFuzz is orthogonal to the methods presented in this study; thus, we can integrate it into *FuzzBuilderEx*.

TABLE 4. Total FA count of target libraries.

Target Library	OSSFuzz	FuzzBuilder (the previous work)	<i>FuzzBuilderEx</i> (this research)
boringssl	5	3	32
c-ares	10	10	21
expat	2	2	7
http-parser	2	2	6
json-c	1	1	14
libhttp	3	3	21
yara	2	2	18

Table 4 presents the number of FAs in the target library for OSSFuzz, FuzzBuilder, and *FuzzBuilderEx*. FuzzBuilder, in a previous work, manually selected FAs in the same way as OSSFuzz, so the number of FAs is the same except for in the library boringssl. For boringssl, the two FAs applied to OSSFuzz are composed of integer-type parameters, so FuzzBuilder excluded the corresponding FAs. FuzzBuilder only supports FAs with a string-type parameter). The results of *FuzzBuilderEx* indicate that the number of FAs increased in all libraries. This result demonstrates that FA selection automation can reduce human expert intervention and expand the fuzzing target.

D. EFFECTIVENESS OF A SEED CORPUS

Each seed corpus was created for FAs selected for each library and compared the line coverage to verify the seed corpus's efficiency automatically generated through *FuzzBuilderEx*. For a fair comparison, we used the same fuzzing executables created by OSSFuzz and applied each seed corpus created by OSSFuzz, FuzzBuilder, and *FuzzBuilderEx*. The FA selection, which is the criterion for seed generation, was also limited to that used by OSSFuzz as FAs for a fair comparison. That is, we experimented under the same conditions, except for the seed corpus. Table 5 lists the information on the FAs for each library used in the experiment. For reference, `ares_parse_*` of c-ares is an abbreviation of the nine FAs starting with `ares_parse_`. Because several fuzzing executables exist for each library, we integrated the results using the gcov tool after fuzzing for each fuzzing executable. Code coverage is based on the line coverage.

Figure 8 presents the comparison results for line coverage. Comparing the results of OSSFuzz, the code coverage of *FuzzBuilderEx* was high in six out of seven libraries.

TABLE 5. FA list of target libraries.

Target Library	FAs
boringssl	CBS_init BIO_new_mem_buf SSL_SESSION_from_bytes
c-ares	ares_create_query ares_parse_*
expat	XML_Parse XML_Parse_buffer
http-parser	http_parser_execute http_parser_parse_url
json-c	json_tokener_parse_ex
libhttp	htp_connp_open htp_connp_req_data htp_connp_res_data
yara	yr_rules_scan_mem yr_compiler_add_string

Compared with the previous study, *FuzzBuilderEx* achieved high coverage in all seven libraries.

Because the developer writes the test code for verification during development, it generally performs various and more complex verifications than the fuzzing test scenarios. Thus, when the FA selected as the fuzzing target and the related function sequence were implemented in the test code, the input values for testing the target library were relatively diverse compared to the fuzzing test scenarios. Therefore, compared with OSSFuzz, *FuzzBuilderEx* achieved high coverage in six out of seven libraries. However, for boringssl, the measured line coverage is lower than that of OSSFuzz, which is related to the FA characteristics and the test code that verifies the FA. In addition, *CBS_init*, the FA used in boringssl, stores values in a specific memory structure defined by the library, and the usage of the memory structure is determined according to which function is called later. However, *FuzzBuilderEx* achieved low coverage because the test code of boringssl has only part of the function sequence for *CBS_init*. Thus, if the test code is added while the library is further developed and the function sequence for *CBS_init* is reinforced, we expect that higher coverage can be achieved using the seed generated in this research.

Compared with the previous study, *FuzzBuilderEx* achieved high coverage in all seven libraries because *FuzzBuilderEx* extended the fuzzing target to include the FA integer-type parameter to perform fuzzing by including more seeds and function sequences than the previous study. However, in c-ares and json-c, a similar line coverage was achieved, as in the previous study, because the test code does not have many function sequences using the integer-type parameter in selected FAs. However, suppose that the test code using the FA integer-type parameter is added when the library is further developed. In this case, we expect that higher coverage can be achieved, similar to other libraries. Lastly, in boringssl, the previous study achieved lower line coverage

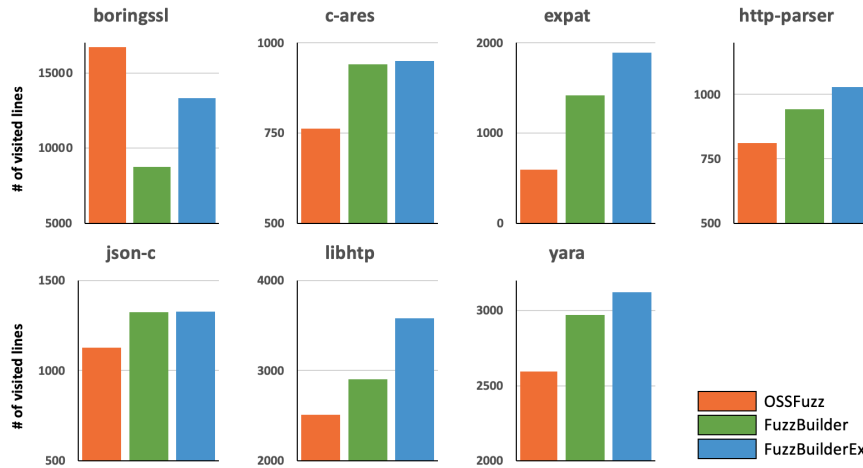


FIGURE 8. Comparison of line coverage for seed corpus. We achieved higher code coverage in 6 libraries excluding boringssl compared to OSSFuzz, and higher performance in all 7 libraries than in the previous study.

than OSSFuzz, but this study could significantly reduce the coverage gap with OSSFuzz owing to the expansion of the FA parameter type.

E. EFFECTIVENESS OF FUZZING EXECUTABLES

This experiment measured the line coverage and unique crash count during fuzzing to verify the fuzzing executable efficiency generated by *FuzzBuilderEx*. For a fair comparison, we used the same seed corpus generated by *FuzzBuilderEx*. As explained in Section VI-D, the FAs of the seven libraries used in the experiment were limited to those included as an FA in OSSFuzz, so the seed corpus created by *FuzzBuilderEx* can be equally applied to OSSFuzz and FuzzBuilder.

Table 6 lists the fuzzing executable of each target library. Fuzzing executables of OSSFuzz were created based on the fuzzing file included in the library directory, and FuzzBuilder created the executable file per FA. FuzzBuilder and *FuzzBuilderEx* have the same number of executable files because they created fuzzing executables based on FAs. For reference, *ares_parser_*_fuzzer* of c-ares is an abbreviation of the nine fuzzing executables starting with *ares_parse_*. Because the number of fuzzing executables is different, this experiment used the AFL parallel mode for a fair comparison. For example, suppose the number of fuzzing executables is two and four, FuzzBuilder and *FuzzBuilderEx*, respectively. In that case, we run AFL parallel mode with four instances to keep the same number of instances in parallel mode even if the number of fuzzing executables differs. In other words, the experiment was performed under the same conditions, except for the fuzzing executable.

Table 7 displays the results of measuring the unique crash count and line coverage for each fuzzing executable created through OSSFuzz, FuzzBuilder, and *FuzzBuilderEx*. As in Section VI-D, we measured seven libraries and compared them with the OSSFuzz results. Excellent results were confirmed for a total of six libraries. However, for boringssl,

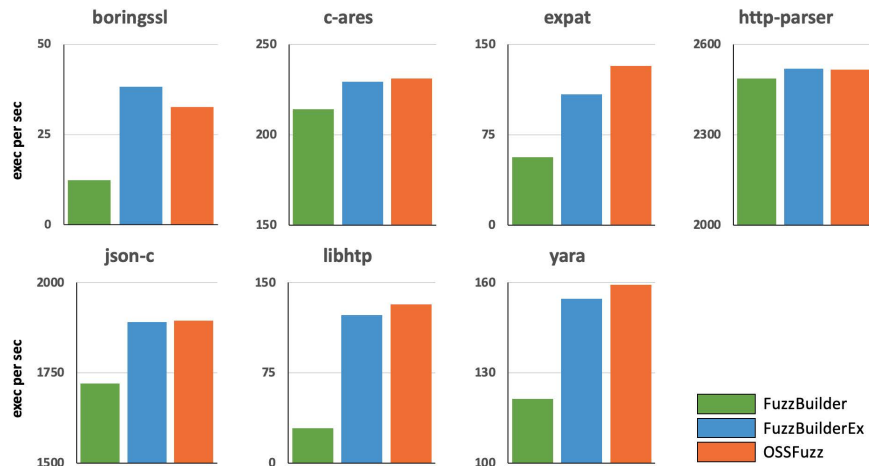
TABLE 6. Fuzzing executable list of target libraries.

Target Library	OSSFuzz	FuzzBuilder(Ex)
boringssl	vn_div vn_mod_exp client dtls_client dtls_server pkcs12 pkcs8 read_pem server session spki ssl_ctx_api	CBS_init_fuzzer BIO_new_mem_buf_fuzzer SSL_SESSION_from_bytes_fuzzer
c-ares	ares_create_query_fuzzer ares_parse_reply_fuzzer	ares_create_query_fuzzer ares_parse_*_fuzzer
expat	parse_ISO_8859_1_fuzzer parse_US_ASCII_fuzzer parse_UTF_16BE_fuzzer parse_UTF_16_fuzzer parse_UTF_16LE_fuzzer parse_UTF_8_fuzzer	XML_Parse_fuzzer XML_Parse_buffer_fuzzer
http-parser	fuzz_parser fuzz_url	http_parser_execute_fuzzer http_parser_parse_url_fuzzer
json-c	tokenr_parser_ex_fuzzer	json_tokener_parse_ex_fuzzer
libhttp	fuzz_htp	http_connnp_open_fuzzer http_connnp_req_data_fuzzer http_connnp_res_data_fuzzer
yara	dex_fuzzer dotnet_fuzzer elf_fuzzer pe_fuzzer rules_fuzzer	yr_rules_scan_mem_fuzzer yr_compiler_add_string_fuzzer

the test code for various function sequences was insufficient for the same reason as in the previous *CBS_init*, so both the coverage and unique crash count were lower than those for OSSFuzz. Additionally, in json-c, the coverage was higher than in OSSFuzz, but the unique crash count was 10, exhibiting the same performance. The test code in json-c repeatedly used a code that determines whether a specific condition has been satisfied using an *ASSERT* statement in the test code.

TABLE 7. Comparison of the line coverage and unique crash count for fuzzing executable.

	boringssl		c-ares		expat		http-parser		json-c		libhttp		yara	
	crash	coverage	crash	coverage	crash	coverage	crash	coverage	crash	coverage	crash	coverage	crash	coverage
OSSFuzz	43	15,208	3	729	2	538	4	763	10	1,173	11	2,351	17	2,565
FuzzBuilder	28	8,292	7	912	10	1,491	4	928	9	1,321	18	2,912	16	2,814
FuzzBuilderEx	39	13,162	9	926	21	2,125	11	1,291	10	1,338	25	3,274	31	3,310

**FIGURE 9.** Comparison of execution speed. Compare to the previous study, *FuzzBuilderEx* improves execution speed in all seven libraries. Compared to *OSSFuzz*, *FuzzBuilderEx* does not reduce execution speed in most libraries.

Thus, we did not include various functional sequences for FAs. However, as test codes are added or reinforced, fuzzing efficiency can increase.

Since the developer writes the test code for verification during development, they generally perform various and more complex verification than the fuzzing test scenarios. Thus, we expect that *FuzzBuilder*'s fuzzing executables, generated based on function sequences in the test code, covered more diverse procedures than *OSSFuzz*'s. In particular, we confirm that *OSSFuzz*'s fuzzing executable includes only basic scenarios that call FAs, while test code-based fuzzing executables include various scenarios using FAs. The experimental result of *c-ares*, *expat*, *http-parser*, *libhttp*, and *yara* in Table 7 shows that various scenarios are effective for fuzzing. Comparing with the previous study results, *FuzzBuilderEx* achieved high performance in the unique crashes and line coverage for all seven libraries. The performance improvement is that the fuzzing efficiency was increased by expanding the FA parameter and optimizing the fuzzing executable. Owing to the expansion of the FA parameter, the fuzzing scope and scenario were expanded to increase the coverage, and the unique crash count may also increase owing to the increased coverage. In addition, by removing unnecessary parts from the fuzzing executable, the fuzzing efficiency increased, and the unique crash count could be increased because it was possible to fuzz a meaningful area more intensively. However, in *c-ares* and *json-c*, the performance of both the unique crash and line coverage was measured similarly for the same reason as discussed in Section VI-D.

Figure 9 depicts the results of measuring the fuzzing execution speed to verify the optimization effect of the fuzzing executables. The experiment employed the execution per second value displayed on the dashboard of the AFL fuzzer. When automatically generating fuzzing executables based on test code, the fuzzing efficiency increases, as shown in Table 7, but they can decrease the speed compared to the *OSSFuzz*'s fuzzing executables that wrote the fuzzing scenario as a manual due to unnecessary functions and heavy size. In particular, the value of the execution per second decreases because the execution speed is slow when the functions that require frequent file access or the input or output of a large file are run. Thus, we expect enhancements in the execution speed by optimizing fuzzing executables, such as eliminating unnecessary functions. However, because the absolute value of the execution per second measured in this experiment may differ depending on the experimental environment, a relative comparison was used to determine whether it improved.

As a result of the experiment, all libraries improved the execution speed compared to *FuzzBuilder*. However, *c-ares* and *http-parser* did not significantly improve the execution speed because the functions eliminated through optimization were only composed of statements that repeated relatively simple operations. In contrast, *libhttp* and *expat* significantly improved the execution speed because the eliminated functions were composed of statements that performed the test data's input or output. The comparison of execution speed with *OSSFuzz* shows that the difference between

OSSFuzz and execution speed is not significant compared to *FuzzBuilderEx*. In particular, in the case of boringssl, we confirmed that the execution speed of the *FuzzBuilderEx* had increased significantly due to removing the source code loading extensive test data that has nothing to do with fuzzing. Moreover, we expect that OSSFuzz has a low execution speed because of including complex fuzzing scenarios. The experimental results, which show high coverage and crashes, in Table 7 prove that. In contrast, in the case of expat, the execution speed of *FuzzBuilderEx* has been improved compared to the existing research, but it has a slightly lower speed than OSSFuzz. As shown in Table 7, both crashes and coverage are low, which is confirmed to be because the OSSFuzz's fuzzing executable is low in complexity. Yara's results were interesting. We can see that despite the relatively higher complexity of *FuzzBuilderEx*'s executable than OSSFuzz's executable; there is no significant reduction in the execution speed. This result is due not only to removing unnecessary code statements in test functions but also to maximizing the fuzzing parallelism through the segmentation of executable files in FA units. In summary, compared to the manually written executable, the test code-based, automatically generated fuzzing executable suffers from lower execution speed. But, this study improves the execution speed by removing unnecessary functions and splitting small-size FA units. Especially considering the code coverage and crash detection performance, we can see that this study minimizes the reduction in fuzzing execution speed, allowing for efficient fuzzing.

F. VULNERABILITY ANALYSIS

To demonstrate the usefulness of *FuzzBuilderEx* in detecting vulnerabilities, we selected four of the nine libraries used in the experiment as the target library for vulnerability analysis. The reason for choosing the library with the most vulnerabilities reported is to determine whether *FuzzBuilderEx* can detect new vulnerabilities that have not been discovered even though many studies have analyzed them.

TABLE 8. Found known vulnerabilities from open-source libraries.

Target Library	Vul. Type	CVE
c-ares	Heap-based buffer overflow	(known) CVE-2016-5180
expat	Heap-based buffer over-read	(known) CVE-2019-15903

We experimented on the old library version that includes known CVEs to evaluate the ability to detect known vulnerabilities. To determine whether already known vulnerabilities can be detected, we selected two libraries, c-ares, and expat, among the target libraries used in Sections VI-D and VI-E. As described in Table 8, we detected two vulnerabilities [58], [59], which were already published in the CVE.

We applied *FuzzBuilderEx* to the mpc and cJSON libraries to evaluate their ability to detect unknown vulnerabilities. The mpc library has already been applied to OSSFuzz, and cJSON is not applied to OSSFuzz. As shown in Table 9,

TABLE 9. Found unknown vulnerabilities from open-source libraries.

Target Library	Vul. Type	CVE
mpc	Null Dereference heap-based buffer over-write	(zero-day) silent-patch (zero-day) silent-patch
cJSON	Improper Check for Unusual or Exceptional Conditions	(zero-day) CVE-2019-1010239

we detected three vulnerabilities. Two vulnerabilities in the mpc library were not registered as CVEs but were delivered to the developer as zero-day vulnerabilities and were silently patched [60] in the next version. Another vulnerability received a CVE ID (CVE-2019-1010239) [61] as zero vulnerability. The two vulnerabilities in mpc are significant. They were detected by only *FuzzBuilderEx* as vulnerabilities and were not detected by OSSFuzz and the previous study, FuzzBuilder.

G. SUMMARY OF EXPERIMENTAL RESULTS

This study experimentally verified the effectiveness of the *FuzzBuilderEx*. First, the extent to which human expert intervention was reduced through the automation method proposed in this study was analyzed by dividing the library fuzzing steps and whether automation is possible for each step. This study significantly improved the automation level compared to the previous study by automating the FA selection, seed corpus, and fuzzing executable generation, which requires human expert intervention.

In this experiment, *FuzzBuilderEx* was applied to seven libraries to evaluate the efficiency of the seed corpus and fuzzing executable. The seed corpus generated automatically in this study has a coverage improvement of 31.2% compared to the previous study because the fuzzing scope has been expanded to the FA parameter. The fuzzing executable also improved by 58.7% based on the unique crash count compared to the previous study. This improvement is due to the FA parameter expansion and fuzzing executable optimization, so that more meaningful code statements can be intensively fuzzed. Moreover, *FuzzBuilderEx* achieved a higher execution speed than the previous study through fuzzing executable optimization, and we confirmed that the reduction in fuzzing speed was not significant compared to OSSFuzz. Finally, by performing a vulnerability analysis, *FuzzBuilderEx* detected five vulnerabilities. Among them, three vulnerabilities were zero-day vulnerabilities. One vulnerability is registered in the CVE. In particular, in the case of the three unknown vulnerabilities detected in this study, note that it is meaningful in that it detects a vulnerability that was not discovered despite massive analysis in many fuzzing studies such as OSSFuzz.

In summary, the experimental results demonstrate that *FuzzBuilderEx* can facilitate library fuzzing by minimizing human expert intervention, and efficient fuzzing is possible by automatically generating a seed corpus and fuzzing executable.

VII. DISCUSSION

Although *FuzzBuilderEx* is widely applicable to library fuzzing and is effective in detecting vulnerabilities, there is room for future improvement. This section describes the limitations of the current *FuzzBuilderEx* and its possible improvements through future research. We then describe some of the lessons learned from this research.

A. COMPLEX FUNCTION SEQUENCE

FuzzBuilderEx automatically generates a fuzzing executable and seed corpus by conducting a static/dynamic analysis of the test code in the target library. However, this research used only the features of the test framework in the generation process. As defined in Section IV-B, the test framework has an execution mode that runs the test code sequentially and randomly runs it. It also has an injection mode for running tests by injecting defects. With the fuzzing executable and seed corpus generated by the various modes of the test framework, we can reproduce more diverse states of a target library. Furthermore, the overall fuzzing performance is improved, and more vulnerabilities can be detected efficiently.

B. EXTENSION OF THE TARGET LANGUAGE

Currently, *FuzzBuilderEx* targets only libraries developed in C/C++. Because this study utilized the test framework and LLVM IR for automation, although it is easily extendable to other program languages, additional research to consider linguistic characteristics is necessary. Moreover, the current design of *FuzzBuilderEx* is limited to the automated generation of fuzzing executables and seed corpus, assuming that the program's source code is available. It is more challenging to automatically generate a fuzzing executable and seed corpus for library fuzzing based on the binary without the source code.

C. SEED AUGMENTATION

FuzzBuilderEx generates a seed corpus based on the test code in the library. Thus, if the test code is insufficient or only a simple test code is present, a lower level of coverage than the existing method can be achieved, such as the experimental result of BoringSSL. *FuzzBuilderEx* can conduct further research that can augment a new seed based on the automatically generated seed by applying a deep learning approach to resolve this problem. In other words, further research might focus on training a deep learning model based on the automatically generated seed using *FuzzBuilderEx* and augment the seed based on the trained model. In addition, these additional studies fit the purpose of this study, which is to provide an automated library fuzzing environment to minimize the need for expert interventions.

We can also apply the test code auto-generated method [62] to automatically generate a test code with sufficient coverage if the test code is missing or insufficient. From a test perspective, it is challenging to automatically create test cases because there is no test oracle, which is a mechanism for

determining whether a test has passed or failed. However, fuzzing does not check the test case pass, so the test oracle is not essential in the case of test code generation for fuzzing. We expect to apply the automated test case generation technique without difficulty.

D. LESSONS LEARNED

This section describes some lessons learned while researching *FuzzBuilderEx* and utilizing it to generate seed corpora and fuzzing executables.

- **Fuzzing executable segmentation not only increases the execution speed but also improves the fuzzing efficiency.** To increase the fuzzing execution speed, this experiment applied parallel fuzzing of the AFL, which allowed multiple fuzzing executables to be executed simultaneously. Initially, we expected to increase the fuzzing execution speed by maximizing parallelism, but it also increases fuzzing efficiency through seed sync of parallel fuzzing, which applies a seed queue with priority to each fuzzing executable. Therefore, we recommend creating several fuzzing executables separately in a logically functional unit rather than creating a single fuzzing executable with multiple functions.

- **High-level tuning for fuzzing efficiency still requires an expert resource.** Although efforts such as eliminating unnecessary test functions and code statements through automation have minimized speed reductions, high levels of speed improvement still require expert intervention for several reasons.: First, even for test functions that include FA, fuzzing is unnecessary if the procedure is simple and it has already been sufficiently validated. For these functions, it is challenging to exclude them through automation, without expert interventions. Therefore, these functions should be explicitly excluded through expert analysis. Second, in this study, to generate fuzzing executables in logical functional units, the test code was retrieved based on FA and configured only with a test code containing FA to form the fuzzing executable. However, the fuzzing executable can be further subdivided logically within the test code, including multiple FAs. This requires additional resources, such as logical flow analysis within the test code, which is difficult to perform through automation. Therefore, a high level of executable segmentation still requires expert analysis.

- **A separate optimization algorithm is required for automatically generated seed corpus through the test code.** As described in Section V-C, AFL-cmin was used for seed corpus optimization. The AFL fuzzer provides AFL-tmin in addition to AFL-cmin as a seed optimizer tool. Unlike AFL-cmin, which eliminates the duplication between seed files, AFL-tmin optimizes by maintaining only valid values within the seed file and removing invalid values [63]. Therefore, in general, when using AFL to optimize the seed, we first optimized the seed corpus with AFL-tmin and then applied AFL-cmin to optimize them. Thus, we applied AFL-tmin and AFL-cmin sequentially, but contrary to expectations, the fuzzing efficiency was reduced after optimizing the seed corpus. This is considered to have a limitation in

selecting a valid part of a seed corpus using only the information obtained from the dry run, as test codes often perform similar tests repeatedly. Therefore, it is necessary to introduce and apply seed optimization algorithms for generated seed corpus through the test code.

VIII. CONCLUSION

Although many security communities have performed research on fuzzing with an automated approach to discover software vulnerabilities, many modern fuzzing methods have not been applied to libraries because fuzzing requires an execution environment, particularly an executable file. To resolve this problem, we proposed *FuzzBuilderEx*, a system that automatically generates a fuzzing executable and a seed corpus for a library based on the test framework. In particular, after proposing a prior study to automatically provide a fuzzing environment for a library using a test framework, we propose a new approach that significantly improves the fuzzing performance and efficiency.

The executable file created in *FuzzBuilderEx* can be extended by applying them to various fuzzers, such as AFL, and it frees developers from the need to write additional source code for fuzzing and to maintain the code. Furthermore, this study enables the tester to perform fuzzing without a high level of knowledge about the target program. In comparison to previous studies, this study evaluated the performance of *FuzzBuilderEx* by applying it to nine open-source libraries. The experimental results showed that *FuzzBuilderEx* has the best performance in terms of fuzzing speed and vulnerability detection ability compared to related studies. These evaluation results indicate that *FuzzBuilderEx* can help strengthen software security by applying fuzzing to various types of libraries without significant effort. Moreover, we detected three unknown vulnerabilities and registered one of them in the CVE [61] as a zero-day vulnerability. Because the source code for *FuzzBuilderEx* is available on GitHub [13], other researchers can reproduce our results and extend the automated generation technology of the fuzzing executable and seed corpus for library fuzzing.

APPENDIX A TERMINOLOGY

In addition to Section IV, Table 10 describes the terms necessary for understanding this study.

APPENDIX B SEED GENERATION ALGORITHM

This section describes algorithms that automatically generate seeds by inserting the LLVM API into the target library. (Refer to algorithm 2) First, the FA information is obtained from the seed configuration file automatically created in the Inferring FA phase. The FA information includes the FA name and parameter information. Then *FuzzBuilderEx* infers the FA in the target library and searches the entry block of the FA where the LLVM IR code is inserted and splits the previous block and the next block based on the entry

TABLE 10. Terminology for FuzzBuilderEx.

Term	Definitions
(Base) Fuzzer	A fuzzer [64] is a tool that performs fuzzing for a program under test (PUT) and triggers crash by applying several algorithms to input data according to the fuzzing method.
Fuzz campaign	A fuzz campaign [42] refers to the execution of a fuzzer to check whether a target program complies with a specific security policy. The purpose of performing a fuzzing campaign on the PUT is to discover defects that violate the specific security policy.
Grey-box fuzzing	Grey-box fuzzing [3] is a technique that performs fuzzing using program execution information, such as control flow and coverage information. AFL is the most representative grey-box fuzzing tool and is a coverage-guided fuzzer based on coverage information. Recently, hybrid fuzzing, which performs fuzzing by adding dynamic information through symbolic execution and taint analysis in addition to coverage information, has been actively researched.
Fuzzable API	A fuzzable API (FA) [12] indicates the library interface functions that are the target of fuzzing. The fuzzer feeds input data to the target library through the FA.
Function sequence	A function sequence [11] indicates a series of function calls to implement a specific feature and is a set of functions to be tested jointly. We cannot thoroughly test a library with a single function and must test all related functions jointly. Thus we regard it as calling a series of functions when writing source code for library fuzzing.
Seed corpus	A seed corpus [65] is a set of files that is the input data for the fuzzer. The fuzzer mutates the initial seed corpus using several algorithms. In general, a seed corpus significantly affects the fuzzing efficiency. Thus, it is crucial to generate a valuable seed corpus that can trigger a unique crash and path.
Fuzz driver	The fuzz driver [10] is a module with the role of executing the library code for library fuzzing. In other words, the fuzz driver helps communicate the fuzzer and target library and runs the library. Most libraries do not have a fuzz driver, so they are not easy to apply. Generally, a fuzzing executable includes a fuzz driver.
Fuzzing executable	The fuzzing executable [66] is an executable file that is the target of fuzzing and includes the fuzz driver. The fuzzing executable communicates with the fuzzer and must operate using the seed corpus fetched from it. We must create the fuzzing executable separately because the library does not have an executable file even after building it.
Fuzzing environment	A fuzzing environment [12] indicates a set of necessary files to perform fuzzing. The fuzzing environment is primarily composed of the fuzzer, fuzzing executables, and seed corpus.
Test case (Test code)	A test case [67] is a series of code sequences to exercise a particular program path or verify compliance with a specific requirement. A test case includes a specification of the inputs, execution conditions, testing procedure, and expected results. This paper uses the broad definition of a test case, including a test code written by the developer.
Unit test	In software engineering, unit testing [68] is a software testing method by which individual units of source code are tested to determine whether they are fit for use. A unit is commonly an individual function or procedure. Because the developer in the development process primarily writes the unit test, it can include more diverse test cases than the tester's fuzzing code.

block. The inserted LLVM IR code functions to write the values that enter the parameter in the file. (Refer to Listing 3) *FuzzBuilderEx* inserts the LLVM IR code for seed generation between the prev block and the next block according to the parameter information of the FA. The above process is

run on all FAs, and we can create a new binary based on the modified LLVM bytecode.

Algorithm 2 Seed Generation

```

1: procedure Inst_for_Seed_Gen(lib, fa_info)
2:   func_list  $\leftarrow$  get_func_list(library)
3:
4:   for func  $\leftarrow$  func_list do
5:     if is_FA(func, FA_info) then
6:       entry  $\leftarrow$  get_entry_block()
7:       prev_block, next_block  $\leftarrow$ 
         split_block(entry)
8:       param_info  $\leftarrow$  get_param_info(FA_info)
9:
10:      if is_target_param(param_info) then
11:        insert_block  $\leftarrow$ 
          create_block(str_ir, int_ir)
12:      else if is_int_param(param_info) then
13:        insert_block  $\leftarrow$  create_block(int_ir)
14:      else if is_str_param(param_info) then
15:        insert_block  $\leftarrow$  create_block(str_ir)
16:      end if
17:    end if
18:
19:    new_block  $\leftarrow$ 
      block_concat(prev_block, insert_block, next_block)
20:    new_library  $\leftarrow$  modify(new_block)
21:  end for
22:
23:  return new_library
24: end procedure

```

APPENDIX C

FUZZING EXECUTABLE GENERATION ALGORITHM

This section describes algorithms for automatically generating fuzzing executables by inserting the LLVM API into the test binary. (Refer to algorithm 3) First, *FuzzBuilderEx* analyzes the test binary and obtains information about the test function list, and the main function corresponding to the entry point of the test binary within the binary. Subsequently, we insert the interface for transferring seed files from the base fuzzer to the target library into the main function, referring to the entry point information. The inserted interface contains code statements that add a global variable to store seed files in the main function and provides seeds to the global variable. Subsequently, it is repeatedly applied to all test functions, and if an FA exists within the test function, the previously inserted global variable is replaced with the parameters of the FA. The LLVM IR code that replaces the parameters of the FA should be configured differently according to the parameter type. Refer to Listings 4 and 5 for the pseudo-code that adds global variables to the main function and replaces parameters of the FA. In addition, removing unnecessary parts within the test function is performed on functions that contain FA, and functions that do not include FA are eliminated. Finally,

we generate a new binary based on the modified LLVM bytecode through the preceding process.

Algorithm 3 Fuzzing Executable Generation

```

1: procedure Inst_for_Exe_Gen(te_bin, fa_info)
2:   test_funcs, entry  $\leftarrow$  preprocessing(test_bin)
3:   param_info  $\leftarrow$  get_param_info(FA_info)
4:
5:   if is_str_and_int_param(param_info) then
6:     entry  $\leftarrow$  insert_str_int_interface(entry)
7:   else if is_int_param(param_info) then
8:     entry  $\leftarrow$  insert_int_interface(entry)
9:   else if is_str_param(param_info) then
10:    entry  $\leftarrow$  insert_str_interface(entry)
11:  end if
12:
13:  for func  $\leftarrow$  test_funcs do
14:    if is_FA_exist(func) then
15:      if is_target_param(param_info) then
16:        new_block  $\leftarrow$ 
          insert_str_int_operand(func)
17:      else if is_int_param(param_info) then
18:        new_block  $\leftarrow$  insert_int_operand(func)
19:      else if is_str_param(param_info) then
20:        new_block  $\leftarrow$  insert_str_operand(func)
21:      end if
22:    else
23:      new_block  $\leftarrow$  remove_func(func)
24:    end if
25:
26:    new_test_bin  $\leftarrow$  modify(new_block)
27:  end for
28:
29:  return new_test_bin
30: end procedure

```

REFERENCES

- [1] B. P. Miller, L. Fredriksen, and B. So, "An empirical study of the reliability of UNIX utilities," *Commun. ACM*, vol. 33, no. 12, pp. 32–44, Dec. 1990.
- [2] AFL: American Fuzzy Lop. Accessed: May 2022. [Online]. Available: <https://lcamtuf.coredump.cx/afl/>
- [3] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based grey-box fuzzing as Markov chain," *IEEE Trans. Softw. Eng.*, vol. 45, no. 5, pp. 489–506, May 2019.
- [4] S. Gan, C. Zhang, X. Qin, X. Tu, K. Li, Z. Pei, and Z. Chen, "ColIAFL: Path sensitive fuzzing," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2018, pp. 679–696.
- [5] H. Peng, Y. Shoshitaishvili, and M. Payer, "T-fuzz: Fuzzing by program transformation," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2018, pp. 697–710.
- [6] I. Yun, S. Lee, M. Xu, Y. Jang, and T. Kim, "QSYM: A practical concolic execution engine tailored for hybrid fuzzing," in *Proc. 27th USENIX Secur. Symp. (USENIX Secur.)*, 2018, pp. 745–761.
- [7] P. Chen and H. Chen, "Angora: Efficient fuzzing by principled search," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2018, pp. 711–725.
- [8] K. Serebryany. *A Library for Coverage-Guided Fuzz Testing (Within LLVM)*. Accessed: May 2022. [Online]. Available: <https://lvm.org/docs/LibFuzzer.html>
- [9] K. Serebryany, "OSS-Fuzz—Google's continuous fuzzing service for open source software," in *Proc. USENIX Secur. Symp.*, 2017.

- [10] D. Babić, S. Bucur, Y. Chen, F. Ivančić, T. King, M. Kusano, C. Lemieux, L. Szekeres, and W. Wang, "FUDGE: Fuzz driver generation at scale," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Aug. 2019, pp. 975–985.
- [11] K. Ispoglou, D. Austin, V. Mohan, and M. Payer, "FuzzGen: Automatic fuzzer generation," in *Proc. 29th USENIX Secur. Symp. (USENIX Secur.)*, 2020, pp. 2271–2287.
- [12] J. Jang and H. K. Kim, "FuzzBuilder: Automated building greybox fuzzing environment for C/C++ library," in *Proc. 35th Annu. Comput. Secur. Appl. Conf.*, 2019, pp. 627–637.
- [13] GitHub. *FuzzBuilderEx*. Accessed: May 2022. [Online]. Available: <https://github.com/kppw99/FuzzBuilderEx>
- [14] D. Aitel, "An introduction to spike, the fuzzer creation kit," in *Proc. Black Hat USA*, 2001, pp. 1–32.
- [15] CERT. *BFF—Basic Fuzzing Framework*. Accessed: May 2022. [Online]. Available: <https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=507974>
- [16] CERT. *FOE—Failure Observation Engine*. Accessed: May 2022. [Online]. Available: <https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=515311>
- [17] S. Hocevar. *Zzuf*. May 2022. [Online]. Available: <https://github.com/samhocevar/zzuf>
- [18] M. Security. *Funfuzz*. Accessed: May 2022. [Online]. Available: <https://github.com/MozillaSecurity/funfuzz>
- [19] M. Eddington. *Peach*. Accessed: May 2022. [Online]. Available: <http://community.peachfuzzer.com/WhatIsPeach.html>
- [20] S. Rawat, V. Jain, A. Kumar, L. Cojocar, C. Giuffrida, and H. Bos, "VUzzer: Application-aware evolutionary fuzzing," in *Proc. NDSS*, vol. 17, 2017, pp. 1–14.
- [21] H. Zhong et al., "MAPO: Mining and recommending API usage patterns," in *Proc. Eur. Conf. Object-Oriented Program.* Berlin, Germany: Springer, 2009.
- [22] M. A. Saied, O. Benomar, H. Abdeen, and H. Sahraoui, "Mining multi-level API usage patterns," in *Proc. IEEE 22nd Int. Conf. Softw. Anal., Evol., Reeng. (SANER)*, Mar. 2015, pp. 23–32.
- [23] J. Fowkes and C. Sutton, "Parameter-free probabilistic API mining across GitHub," in *Proc. 24th ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, Nov. 2016, pp. 254–265.
- [24] R. P. L. Buse and W. Weimer, "Synthesizing API usage examples," in *Proc. 34th Int. Conf. Softw. Eng. (ICSE)*, Jun. 2012, pp. 782–792.
- [25] J. E. Montandon, H. Borges, D. Felix, and M. T. Valente, "Documenting APIs with examples: Lessons learned with the apiminer platform," in *Proc. 20th Work. Conf. Reverse Eng. (WCORE)*, Oct. 2013, pp. 401–408.
- [26] L. Moreno, G. Bavota, M. Di Penta, R. Oliveto, and A. Marcus, "How can i use this method?" in *Proc. IEEE/ACM 37th IEEE Int. Conf. Softw. Eng.*, May 2015, pp. 880–890.
- [27] N. Katirtzis, T. Diamantopoulos, and C. A. Sutton, "Summarizing software API usage examples using clustering techniques," in *Proc. FASE*, 2018, pp. 189–206.
- [28] C. Pacheco, S. K. Lahiri, M. D. Ernst, and T. Ball, "Feedback-directed random test generation," in *Proc. 29th Int. Conf. Softw. Eng. (ICSE)*, May 2007, pp. 75–84.
- [29] W. Zheng, Q. Zhang, M. Lyu, and T. Xie, "Random unit-test generation with MUT-aware sequence recommendation," in *Proc. IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Sep. 2010, pp. 293–296.
- [30] S. Zhang, D. Saff, Y. Bu, and M. D. Ernst, "Combined static and dynamic automated test generation," in *Proc. Int. Symp. Softw. Test. Anal. (ISSTA)*, 2011, pp. 353–363.
- [31] M. Pradel and T. R. Gross, "Leveraging test generation and specification mining for automated bug detection without false positives," in *Proc. 34th Int. Conf. Softw. Eng. (ICSE)*, Jun. 2012, pp. 288–298.
- [32] C. Lyu, S. Ji, Y. Li, J. Zhou, J. Chen, and J. Chen, "SmartSeed: Smart seed generation for efficient fuzzing," 2018, *arXiv:1807.02606*.
- [33] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," 2014, *arXiv:1406.2661*.
- [34] J. Jiang, H. Xu, and Y. Zhou, "RULF: Rust library fuzzing via API dependency graph traversal," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Nov. 2021, pp. 581–592.
- [35] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "AC/C++ code vulnerability dataset with code changes and CVE summaries," in *Proc. 17th Int. Conf. Mining Softw. Repositories*, 2020, pp. 508–512.
- [36] K. Beck. *Simple Smalltalk Testing: With Patterns*. Accessed: May 2022. [Online]. Available: <https://web.archive.org/web/20150315073817/http://www.xprogramming.com/testfram.htm>
- [37] T. J. Team. *JUnit5—The 5th Major Version of the Programmer-Friendly Testing Framework for Java and the JVM*. Accessed: May 2022. [Online]. Available: <https://junit.org/junit5/>
- [38] S. B. Charlie Poole, R. Prouse, and N. Colvin. *NUnit—A Unit-Testing Framework for All .Net Languages*. Accessed: May 2022. [Online]. Available: <https://nunit.org/>
- [39] M. Feathers. *CPPUnit Test Framework*. Accessed: May 2022. [Online]. Available: <https://www.freedesktop.org/wiki/Software/cppunit/>
- [40] Google. *Googletest—Google Testing and Mocking Framework*. Accessed: May 2022. [Online]. Available: <https://github.com/google/googletest>
- [41] GitHub. *GitHub Ranking*. Accessed: May 2022. [Online]. Available: <https://github.com/EvanLi/Github-Ranking>
- [42] V. J. M. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *IEEE Trans. Softw. Eng.*, vol. 47, no. 11, pp. 2312–2331, Nov. 2021.
- [43] J. Wang, B. Chen, L. Wei, and Y. Liu, "Skyfire: Data-driven seed generation for fuzzing," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 579–594.
- [44] H. Zhu, P. A. V. Hall, and J. H. R. May, "Software unit test coverage and adequacy," *ACM Comput. Surv.*, vol. 29, no. 4, pp. 366–427, 1997.
- [45] LLVM. *LLVM Compiler Infrastructure*. Accessed: May 2022. [Online]. Available: <https://llvm.org/docs/>
- [46] NIST. *CWE Top25*. Accessed: May 2022. [Online]. Available: https://cwe.mitre.org/top25/archive/2021/2021_cwe_top25.html
- [47] J. Metzman et al., "FuzzBench: An open fuzzer benchmarking platform and service," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021.
- [48] Open-Source. *BoringSSL*. Accessed: May 2022. [Online]. Available: <https://boringssl.googlesource.com/boringssl>
- [49] C-Ares. Accessed: May 2022. [Online]. Available: <https://github.com/c-ares/c-ares>
- [50] Expat. Accessed: May 2022. [Online]. Available: <https://github.com/libexpat/libexpat>
- [51] HTTP-Parser. Accessed: May 2022. [Online]. Available: <https://github.com/nodejs/http-parser>
- [52] Json-C. Accessed: May 2022. [Online]. Available: <https://github.com/json-c/json-c>
- [53] Libhttp. Accessed: May 2022. [Online]. Available: <https://github.com/OISF/libhttp>
- [54] Yara. <https://github.com/VirusTotal/yara>, Accessed: May 2022.
- [55] MPC. Accessed: May 2022. [Online]. Available: <https://github.com/clsid2/mpc-hc/releases>
- [56] cJSON. Accessed: May 2022. [Online]. Available: <https://github.com/DaveGamble/cJSON>
- [57] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A fast address sanity checker," in *Proc. USENIX Annu. Tech. Conf.*, 2012, pp. 309–318.
- [58] CVE-2016-5180. Accessed: May 2022. [Online]. Available: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2016-5180>
- [59] CVE-2019-15903. Accessed: May 2022. [Online]. Available: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-15903>
- [60] MPC. *Heap Buffer Overflow is Expected-103*. Accessed: May 2022. [Online]. Available: <https://github.com/orangeduck/mpc/issues/103>
- [61] CVE-2019-1010239. Accessed: May 2022. [Online]. Available: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-1010239>
- [62] A. Saswat, E. K. Burke, T. Y. Chen, and J. Clark, "An orchestrated survey of methodologies for automated software test case generation," *J. Syst. Softw.*, vol. 86, no. 8, pp. 1978–2001, 2013.
- [63] S. Pailoor, A. Aday, and S. Jana, "MoonShine: Optimizing OS fuzzer seed selection with trace distillation," in *Proc. 27th USENIX Secur. Symp. (USENIX Secur.)*, 2018, pp. 729–743.
- [64] Y. Chen, Y. Jiang, F. Ma, J. Liang, M. Wang, C. Zhou, Z. Su, and X. Jiao, "EnFuzz: Ensemble fuzzing with seed synchronization among diverse fuzzers," 2018, *arXiv:1807.00182*.
- [65] A. Herrera, H. Gunadi, L. Hayes, S. Magrath, F. Friedlander, M. Sebastian, M. Norrish, and A. L. Hosking, "Corpus distillation for effective fuzzing: A comparative evaluation," 2019, *arXiv:1905.13055*.
- [66] E. Güler, C. Aschermann, A. Abbasi, and T. Holz, "AntiFuzz: Impeding fuzzing audits of binary executables," in *2Proc. 8th USENIX Secur. Symp. (USENIX Secur.)*, 2019, pp. 1931–1947.

- [67] WIKIPEDIA. *Test Case*. Accessed: May 2022. [Online]. Available: https://en.wikipedia.org/wiki/Test_case
- [68] P. Runeson, "A survey of unit testing practices," *IEEE Softw.*, vol. 23, no. 4, pp. 22–29, Jul. 2006.



SANGHOON JEON received the B.S. degree in computer science from Korea University, in 2006, and the M.S. degree in computer engineering from Sungkyunkwan University, in 2011. He is currently pursuing the Ph.D. degree with the School of Cybersecurity, Korea University. He has been a Security Engineer at Samsung Electronics, since 2006. His research interests include embedded system security and data-driven vulnerability/malware detection.



MINSOO RYU received the B.E. degree in computer and information security from Sejong University, in 2020. He is currently pursuing the master's degree with the School of Cybersecurity, Korea University. His recent research interests include fuzzing and app security in various environments.



DONGYOUNG KIM received the B.S. degree in information security from Academic Credit Bank System, in 2019. He is currently pursuing the master's degree with the School of Cybersecurity, Korea University. His recent research interests include fuzzing and vehicle security.



HUY KANG KIM (Member, IEEE) received the B.S. degree in industrial management, the M.S. degree in industrial engineering, and the Ph.D. degree in industrial and systems engineering from the Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 1998, 2000, and 2009, respectively. He founded A3 Security Consulting, the first information security consulting company in South Korea, in 1999. Also, he was a member and the last Leader of KUS (KAIST UNIX Society), the Legendary Hacking Group, South Korea. Currently he is a Professor with the School of Cybersecurity, Korea University. His recent research interest includes solving many security problems in online games based on the user behavior analysis. Before joining Korea University, he was the Technical Director (TD) and the Head of the Information Security Department, NCSOFT, from 2004 to 2010, one of the most famous MMORPG companies in the world.

...